

## Quiz : API Optional (JDK 9+)

### 1. Quelle est la principale utilité de la classe Optional ?

- **A) Éviter les NullPointerException**
  - B) Remplacer tous les types primitifs
  - C) Fournir une alternative à la gestion manuelle des null
  - D) Remplacer les collections en Java
- 

### 2. Que fait la méthode ifPresent(Consumer action) ?

- **A) Exécute action si la valeur est présente**
  - B) Retourne la valeur présente
  - C) Jette une exception si la valeur est absente
  - D) Exécute action quelle que soit la présence
- 

### 3. Quelle est la différence entre ifPresent() et ifPresentOrElse() ?

- A) ifPresent() accepte deux paramètres
  - **B) ifPresentOrElse() permet de définir une action si la valeur est absente**
  - C) ifPresentOrElse() ne fonctionne qu'avec des Streams
  - D) Les deux méthodes font exactement la même chose
- 

### 4. La méthode or(Supplier<? extends Optional<? extends T>> supplier) permet :

- A) De retourner un Optional vide
  - B) D'utiliser une autre valeur si l'Optional courant est vide
  - **C) D'écrire des alternatives chaînables**
  - D) De convertir un Optional en Stream
- 

### 5. À quoi sert la méthode stream() dans Optional (depuis Java 9) ?

- **A) Convertir un Optional en Stream contenant 0 ou 1 élément**
  - B) Transformer une liste en Optional
  - C) Appliquer un filtrage sur un flux d'Optional
  - D) Lire un fichier
- 

### 6. Quel est le résultat du code suivant ?

```
Optional<String> name = Optional.ofNullable(null);
```

```
name.ifPresent(n -> System.out.println(n));
```

- A) Affiche null
  - **B) N'affiche rien**
  - C) Provoque une exception
  - D) Affiche Optional.empty
- 

## 7. Et celui-ci ?

```
Optional<String> name = Optional.ofNullable(null);
name.ifPresentOrElse(
    n -> System.out.println(n),
    () -> System.out.println("Nom non fourni")
);
```

- **A) Affiche Nom non fourni**
  - B) N'affiche rien
  - C) Provoque une exception
  - D) Affiche null
- 

## 8. Quel code utilise or() correctement ?

```
Optional<String> result = Optional.<String>empty().or(() ->
Optional.of("Default"));
System.out.println(result.get());
```

- **A) Affiche Default**
  - B) Affiche Optional.empty
  - C) Provoque une exception
  - D) Ne compile pas
- 

## 9. Expliquez en une phrase l'intérêt d'utiliser Optional.stream() dans le contexte de traitement de collections.

**Permet de transformer un Optional en Stream (0 ou 1 élément), ce qui facilite son intégration dans un pipeline de Stream sans avoir à tester manuellement s'il contient une valeur.**

---

## 10. Complétez ce code :

```
Optional<Integer> value = Optional.ofNullable(42);
value._____;
```

**Réponse :**

```
value.ifPresentOrElse(  
    v -> System.out.println("Valeur présente : " + v),  
    () -> System.out.println("Aucune valeur")  
);
```